# ForgeGIS™

## *GPU Geospatial Compute You Can Check — for Restricted Mission Environments*

GPU-accelerated geospatial compute that travels as Java archives: no GDAL, no system-level native install, and no native binaries of ForgeGIS's own to sign. 239 operations — line of sight, terrain, hydrology, bathymetry, imagery, geometry, point clouds — callable from a script or an AI agent with the same result. Release 1.2.6 puts every measurement on the WGS84 ellipsoid and checks outputs against GDAL, with the scope of every figure stated, so an evaluator can test the claims rather than take them.

**0**

**ForgeGIS native binaries to approve**

*jars on a host that already has a JDK and a GPU driver; no GDAL at runtime*

**WGS84**

**behind every measurement**

*one geodesic engine (Karney's method) for every length, area, distance and bearing; each result names its model*

**130**

**raster ops under contract**

*each held to CPU/GPU agreement, a NoData sweep and a US-survey-foot check in the test suite*

### Deploys as jars

No GDAL, no native install step, no shared libraries of its own: the GPU is reached through the JOCL OpenCL bindings, whose bridge ships inside their jar, and the OpenCL runtime comes with the GPU driver. A JDK and a driver are what the host must already provide. File access is confined to configured read and write roots, and the jar that runs in a lab is the jar that runs on the restricted host.

### Line of sight, on the GPU

Viewshed, batched multi-observer viewshed, intervisibility, Fresnel-zone clearance, line-of-sight profiles, horizon angle and insolation run GPU-resident and compose into pipelines. New in 1.2.6, the observer sits exactly where it was given, viewshed no longer steps over thin ridges, and radius and mast height are measured in true metres on any grid — including US-survey-foot DEMs.

### Same answer, any caller

The agent interface is a parameter-marshalling layer over the same implementations the Java API calls, so an agent-invoked result is the scripted one. Random methods default to a fixed seed, tiled runs are byte-identical to untiled, and benchmarked operations are re-run and compared for determinism. CPU and GPU agree within each operation's declared tolerance.

**Land, water and sea floor in one catalog.** Terrain derivatives and landform classification; hydrology from sink fill to watersheds and height above nearest drainage; bathymetric position, benthic classification, depth uncertainty and ocean acoustics — sound-velocity profiles, the SOFAR channel, surface ducts; vertical datum conversion with the EGM96 geoid bundled, so nothing has to be downloaded on a closed network; LiDAR ground filtering, canopy height and classification; routing and trajectory analysis.



CONTINUED ON REVERSE →

### CHECKED AGAINST GDAL — WITH THE SCOPE STATED

| CHECK | RESULT | SCOPE |
|---|---|---|
| **Viewshed vs `gdal_viewshed`** | 99.656 % of cells agree | a 512² Copernicus 30 m crop, 50 m observer |
| **Contours vs `gdal_contour`** | 599 lines, 17,225 vertices within 1 cm | two SRTM fixtures with NoData voids |
| **Fill NoData vs GDAL invdist** | 456 of 462 filled cells identical | a real-DEM fixture with voids |
| **Ground filter vs PDAL PMF** | 99.597 % of points agree | a USGS 3DEP LiDAR clip |

The viewshed figure is agreement, not identity: ForgeGIS samples each sight line on a bilinear surface and GDAL sweeps line by line. The evidence of record is an exact ray trace of every sight line, gated in the build — ForgeGIS never reports a cell hidden where the surface shows it.

### A VALIDATION FRAMEWORK AN EVALUATOR CAN INTERROGATE

## *A reference chosen per operation class*

GDAL for raster operations with a direct equivalent; JTS for planar topology; GeographicLib and PROJ for geodesy; PDAL for point clouds; scipy, scikit-learn and PyKrige for numerical work; an exact ray trace for visibility; closed-form fixtures where no reference exists. Where an engine and its reference differ by design, the difference is written down as a known divergence rather than tuned away.

## *Three questions, answered separately*

Is the operation correct? (its reference.) Does it give the same answer however it is invoked? (one implementation behind every surface, and CPU/GPU contracts.) Has validation kept pace with the catalog? (a new operation fails the build until it declares its NoData rule and its CPU/GPU tolerance.)

> **Performance — measured on 1.2.6, October 2026.** On one laptop with an RTX 5070, warm medians, in the canonical benchmark run: multi-stage pipelines ran 11–182× ahead of the same chain as GDAL command-line tools at 4096² rasters, and operations with a GDAL equivalent 15× on geometric mean over 242 records on rasters from 16×16 to 4096×4096 and real elevation tiles — 239 faster, 3 slower. Small single operations are not claimed: the 3 that lost were all at 1024² or below. The benchmark's timing method was tightened in this release, so earlier published figures are not directly comparable.

> **Changes in 1.2.6 a program must act on.** Viewshed output is UINT8 — 255 visible, 0 not visible, DEM voids tagged NoData — as `gdal_viewshed` writes it; point parameters default to EPSG:4326 and a point off the raster is refused rather than clamped; `compute_haversine_distance` is now `compute_geodesic_distance`, with no alias.

## Request the Technical Brief and an evaluation build

The Technical Brief covers the accuracy evidence and its scope, the capability catalog, benchmark methodology and deployment. ForgeGIS is proprietary and licensed by written agreement; an evaluation build is available to qualified programs.

rich@seaglassfoundry.com · seaglassfoundry.com

ForgeGIS™, ForgeMind™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. GDAL, PDAL, JTS, GeographicLib, PROJ, Copernicus, SRTM, OpenCL, NVIDIA, GeForce, RTX, Java, and the Model Context Protocol are trademarks or marks of their respective owners; reference to them does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.